

SENTINELNET – AI-POWERED NETWORK INTRUSION DETECTION SYSTEM (NIDS)

Author: Upasana Prabhakar

Mentor: Dr. N Jagan Mohan

ABSTRACT: SentinelNet aims to build a smart Network Intrusion Detection System (NIDS) powered by AI and machine learning. The central goal is to automatically identify and respond to suspicious or potentially harmful network traffic and cyber-attacks in real time. Using historical data, machine learning models will be trained to distinguish between normal and malicious activities, extract crucial features, classify threats, and trigger alerts to help network defenders respond quickly and effectively.

KEYWORDS: Intrusion Detection, Cybersecurity, Machine Learning, AI, Network Traffic

STATEMENT OF ORIGINALITY: This paper presents SentinelNet, a hybrid IDS combining signature-based and anomaly-based methods with advanced preprocessing, feature selection, and class-imbalance handling, applied to NSL-KDD and CICIDS2017 datasets.

1 INTRODUCTION

SentinelNet aims to build a smart Network Intrusion Detection System (NIDS) powered by AI and machine learning. The central goal is to automatically identify and respond to suspicious or potentially harmful network traffic and cyber-attacks in real time.

Using historical data, machine learning models will be trained to:

- Distinguish between normal and malicious activities
- Extract crucial features
- Classify threats
- Trigger alerts to help network defenders respond quickly and effectively

2 LITERATURE

The field of Intrusion Detection Systems (IDS) has evolved from rule-based methods to AI-powered frameworks. Literature highlights:

- **Signature-Based IDS:** Relies on predefined attack signatures, effective for known threats but fails on zero-day attacks.
- **Anomaly-Based IDS:** Uses statistical models and ML to detect unusual traffic patterns, can detect unknown threats but may have higher false positives.

2.1 Benchmark datasets shaping IDS research

- KDD Cup 1999: Early dataset, criticized for redundancy and outdated attacks.
- NSL-KDD: Improved version, duplicates removed, balanced dataset for academic evaluation.
- CICIDS2017: Realistic enterprise traffic, modern attacks like DDoS, infiltration, and web exploits.

2.2 ML algorithms widely used

Decision Trees, Random Forests, SVMs, Neural Networks, Deep Learning, Ensemble methods, Hybrid IDS models.

2.3 Insights

- Feature selection and preprocessing are critical.
- Class imbalance is a challenge.
- Hybrid approaches combine speed of signature-based and intelligence of anomaly-based IDS.

3 METHODOLOGY

3.1 Database

3.1.1 NSL-KDD Dataset

A refined dataset to reduce redundancy:

- 41 features: basic (protocol, service, bytes) + traffic-level (flags, errors)
- Training: 125,973 records
- Test: 22,544 records
- Attack types: DoS (smurf, neptune), Probe (portsweep, nmap), U2R (buffer overflow, rootkit), R2L (ftp write, guess passwd)

3.1.2 CICIDS2017 Dataset

Realistic enterprise network traffic:

- 78–83 features: flow, statistical, packet-level
- Over 2.8 million labeled flow records across 5 days
- Attack types: DDoS (Hulk, GoldenEye, Slowloris), Brute Force, Web Attacks (SQLi, XSS), Botnet/Infiltration, Reconnaissance (PortScan, Heartbleed)

3.1.3 Dataset Comparison

Table 1: Dataset Comparison

Attribute	NSL-KDD	CICIDS2017
No. of Features	41	78–83
Total Records	~150,000	>2.8 million
Traffic	Simulated, old-generation	Realistic, enterprise-scale
Usage	Lightweight model evaluation	Practical deployment
Attack Variety	4 legacy categories	14+ modern attacks

3.2 Preprocessing

The preprocessing pipeline applied in SentinelNet ensures clean, balanced, and representative data for ML models. Steps include:

- Handling missing values using imputation or removal of corrupted records.
- Feature scaling with MinMaxScaler and StandardScaler depending on algorithm needs.
- Encoding categorical features with One-Hot Encoding for protocol types, services, and flags.
- Correlation analysis to detect redundant features; highly correlated attributes were dropped to prevent multicollinearity.
- Addressing class imbalance with:
 - **Power sampling** to balance majority/minority classes.
 - **Undersampling** majority classes where applicable, to prevent bias towards normal traffic.
- Stratified train-test split to preserve class distributions.

3.2.1 Preprocessing Techniques Not Used

Some common preprocessing methods were considered but not applied in this work:

- **Min–Max Normalization:** Not used since standardization (z-score scaling) provided better handling of outliers and ensured feature comparability.
- **SMOTE (Synthetic Minority Oversampling Technique):** Avoided to prevent introducing synthetic traffic samples that might not reflect realistic attack behavior.
- **Label Encoding:** Not applied because categorical variables had no ordinal relationship; one-hot encoding was more appropriate.
- **Log Transformation:** Skipped since most continuous features did not exhibit heavy skewness requiring transformation.

By excluding these methods, the preprocessing pipeline remained tailored to the dataset characteristics while avoiding unnecessary complexity.

3.2.2 Scaling vs Normalization

Table 2: Scaling vs Normalization

Aspect	Standardization	Normalization
Definition	Transforms features to mean=0, std=1	Rescales to [0,1]
Purpose	For Gaussian-based algorithms	For bounded-input algorithms
Use Cases	SVM, Logistic Regression, PCA	Neural Nets, KNN
Effect on Outliers	Less sensitive	Sensitive (compressed to 0–1)
Preserves Data Shape	Yes	Yes, but centers differently
When to Use	Gaussian-based models	NN, image data

3.3 Proposed Method: Logistic Regression

Logistic Regression is a model that predicts the probability of a data point belonging to a class using a simple formula:

$$P = \frac{1}{1 + e^{-z}}, \quad \text{where } z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Here, x_i are input features, w_i are weights, and b is the bias. The output probability is then converted to a class label (commonly using 0.5 as the threshold). Logistic Regression is simple, interpretable, and works well for linearly separable data, but may underperform for complex non-linear patterns in IDS.

4 RESULTS

4.1 NSL-KDD Results

4.1.1 Training vs Testing Accuracy

Table 3: NSL-KDD: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	96.08	77.56
Decision Tree	99.97	79.03
Random Forest	99.97	80.60
Gradient Boosting	98.52	77.95

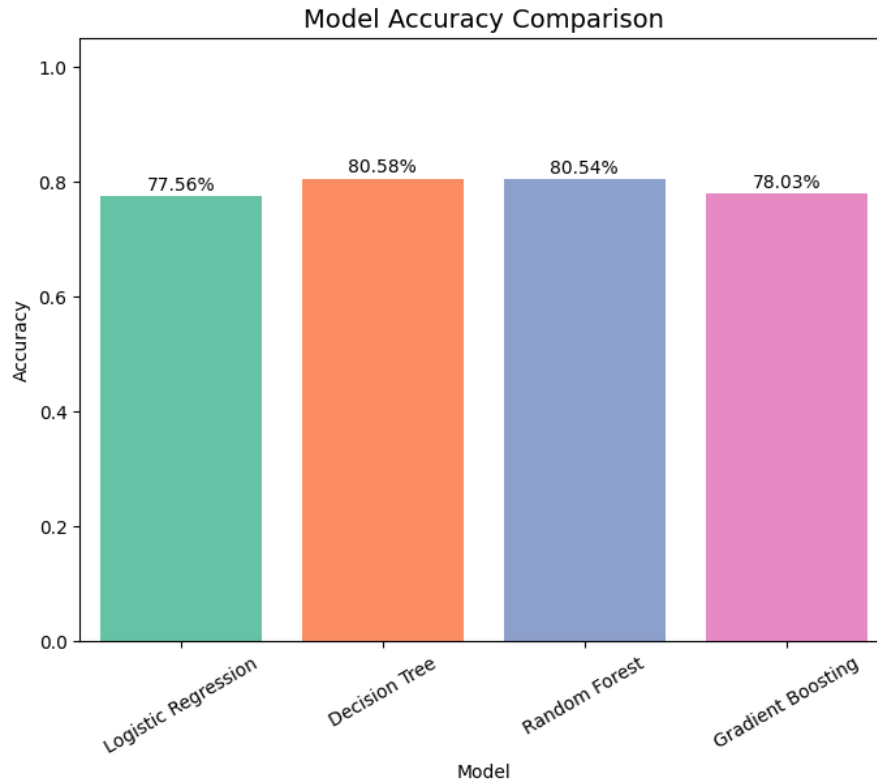


Figure 1: NSL-KDD: Model accuracy comparison showing that Random Forest slightly outperforms other models, while Logistic Regression performs lowest, highlighting the difference in handling nonlinear attack traffic.

4.1.2 Macro Average Performance

Table 4: NSL-KDD: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.80	0.79	0.78
Decision Tree	0.81	0.81	0.79
Random Forest	0.83	0.83	0.81
Gradient Boosting	0.80	0.80	0.78

4.1.3 Macro Average Specificity and Time Taken

Table 5: NSL-KDD: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9634	0.1163
Decision Tree	0.9930	3.1031
Random Forest	0.9954	33.4510
Gradient Boosting	0.9935	56.9905

4.1.4 Confusion Matrix Explanation

The confusion matrix provides detailed insights into classification:

Table 6: Structure of a Confusion Matrix

	Predicted Normal	Predicted Attack
Actual Normal	True Negative (TN)	False Positive (FP)
Actual Attack	False Negative (FN)	True Positive (TP)

Interpretation:

- TN: Correctly identified benign traffic.
- FP: Benign traffic misclassified as attack (false alarm).
- FN: Attack traffic missed (dangerous miss).
- TP: Correctly detected attack.

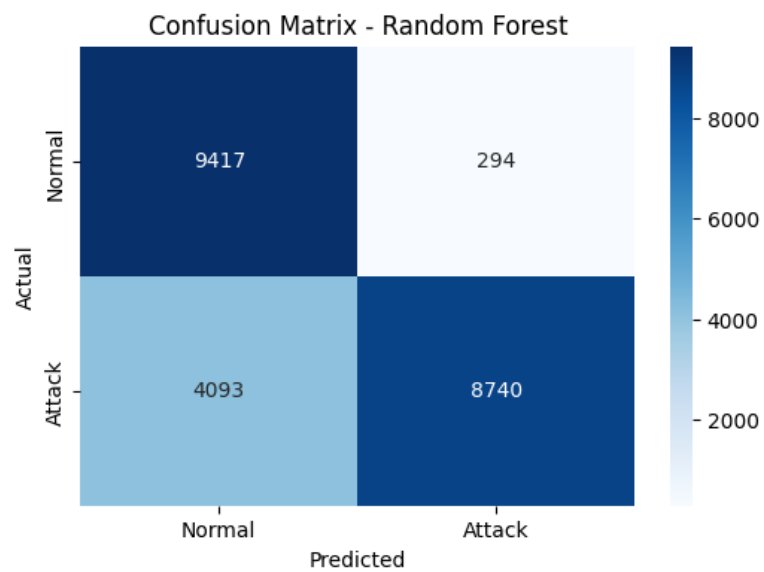


Figure 2: NSL-KDD: Confusion matrix visualization showing classification distribution, with emphasis on how well the models minimize false positives and false negatives.

4.1.5 ROC Curve

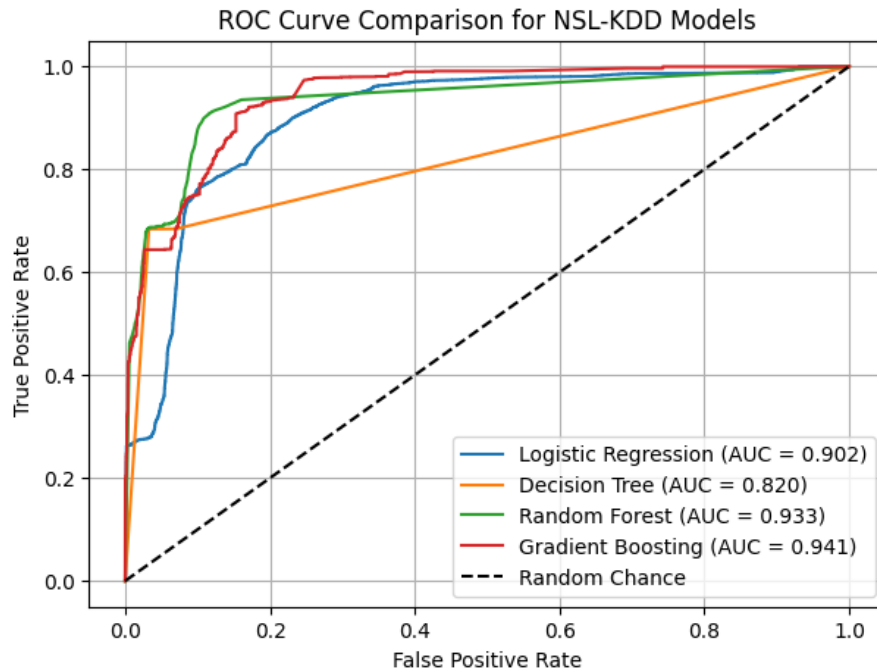


Figure 3: NSL-KDD: ROC curves showing the trade-off between sensitivity and specificity for each model, with Random Forest achieving the best balance.

4.1.6 Model Comparison Analysis

The NSL-KDD dataset shows that Random Forest achieves the best balance between accuracy and F1-score, while Gradient Boosting slightly underperforms compared to expectations. Logistic Regression remains interpretable but limited for complex non-linear attack detection.

4.2 CICIDS2017 Results

4.2.1 Training vs Testing Accuracy

Table 7: CICIDS2017: Training and Testing Accuracies

Model	Training Accuracy (%)	Testing Accuracy (%)
Logistic Regression	96.92	97.04
Decision Tree	99.06	99.08
Random Forest	99.78	99.77
Gradient Boosting	99.83	99.83

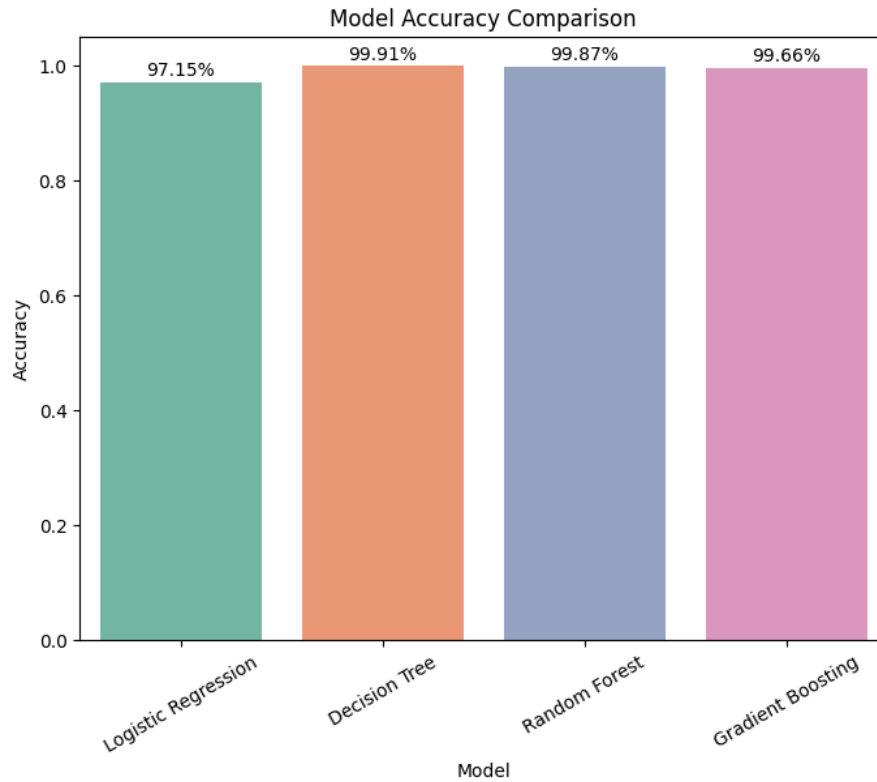


Figure 4: CICIDS2017: Model accuracy comparison showing all ensemble methods achieve near-perfect accuracy, clearly outperforming Logistic Regression.

4.2.2 Macro Average Performance

Table 8: CICIDS2017: Macro Average Precision, Recall, F1

Model	Precision	Recall	F1-score
Logistic Regression	0.96	0.97	0.97
Decision Tree	0.99	0.99	0.99
Random Forest	1.00	1.00	1.00
Gradient Boosting	1.00	1.00	1.00

4.2.3 Macro Average Specificity and Time Taken

Table 9: CICIDS2017: Macro Average Specificity and Time Taken (s)

Model	Specificity	Time Taken (s)
Logistic Regression	0.9933	5.4383
Decision Tree	0.9993	15.2786
Random Forest	0.9990	90.6149
Gradient Boosting	0.9991	338.5894

4.2.4 Confusion Matrix

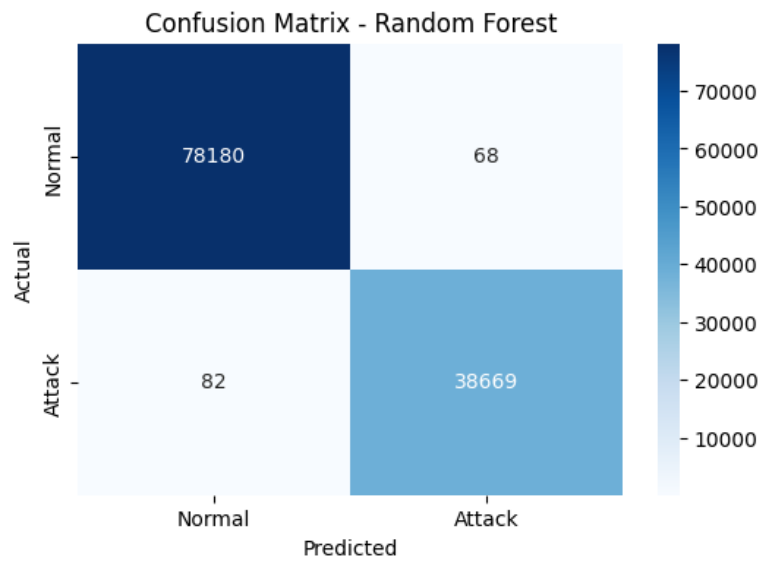


Figure 5: CICIDS2017: Confusion matrix visualization showing that ensemble models nearly eliminate false negatives and false positives, indicating high robustness.

4.2.5 ROC Curve

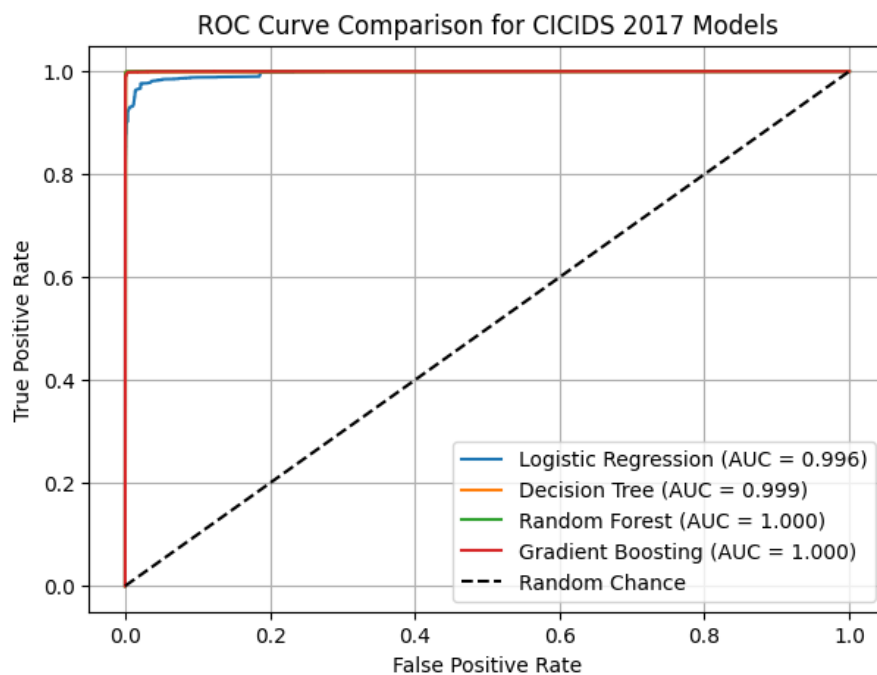


Figure 6: CICIDS2017: ROC curves displaying almost perfect classification capability for Random Forest and Gradient Boosting, closely followed by Decision Tree and Logistic Regression.

4.2.6 Model Comparison Analysis

The CICIDS2017 dataset confirms that ensemble methods (Random Forest, Gradient Boosting) perform nearly perfectly, reflecting their robustness on modern attack traffic. Logistic Regression, while performing well, lags slightly due to its linear assumptions.

4.3 NSL-KDD Final Results

Table 10: NSL-KDD: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.66	0.9634	0.92	0.77	0.902	96.08%	96.01%	77.56%	0.1163
Decision Tree	0.66	0.9930	0.96	0.78	0.807	99.97%	99.30%	79.21%	3.1031
Random Forest	0.68	0.9954	0.97	0.80	0.936	99.97%	99.43%	80.56%	33.4510
Gradient Boosting	0.66	0.9935	0.93	0.77	0.941	98.52%	98.17%	78.03%	56.9905

4.4 CICIDS2017 Final Results

Table 11: CICIDS2017: Final Performance Metrics

Model	Sensitivity	Specificity	Precision	F1-Score	AUC	Train Acc	Val Acc	Test Acc	Train Time (s)
Logistic Regression	0.98	0.9933	0.94	0.96	0.996	96.92%	97.21%	97.04%	5.4383
Decision Tree	0.98	0.9993	0.99	0.99	0.996	99.06%	99.92%	99.08%	15.2786
Random Forest	1.00	0.9990	1.00	1.00	1.000	99.78%	99.86%	99.77%	90.6149
Gradient Boosting	1.00	0.9991	1.00	1.00	1.000	99.83%	99.70%	99.83%	338.5894

5 DISCUSSION

The results demonstrate that tree-based models like Random Forest and Gradient Boosting consistently outperform linear models such as Logistic Regression, both on NSL-KDD and CICIDS2017.

5.1 Key Insights

- NSL-KDD remains useful for academic benchmarking but lacks modern attack representation.
- CICIDS2017 better reflects real-world attack traffic, showing the potential of ML for deployment.
- Handling imbalance and rare classes remains critical.

6 CONCLUSION

SentinelNet demonstrates how AI-powered approaches can significantly enhance intrusion detection. The use of hybrid detection techniques, advanced preprocessing, and careful model evaluation allows for accurate and scalable intrusion detection systems. While results are promising, handling class imbalance and detecting low-frequency attack types remain open challenges.

7 FUTURE WORK

- Incorporating deep learning (CNNs, RNNs, Transformers) for sequential traffic patterns.
- Exploring online learning for real-time adaptation to evolving threats.
- Enhancing explainability of AI models to aid cybersecurity analysts.
- Extending evaluation to encrypted traffic analysis.
- Integrating SentinelNet with SIEM (Security Information and Event Management) systems.

8 ACKNOWLEDGEMENTS

We would like to thank our mentor, project guides, and dataset providers for their support throughout this research. Special thanks go to the Infosys Springboard initiative for providing the opportunity to develop and present this project. We also acknowledge the open-source research community for making datasets like NSL-KDD and CICIDS2017 publicly available, which made experimentation possible.